

Programação Mobile

TextField e TextFormField

Professor: Joseph Donald

🌐 Contatos: www.josephdonald.wordpress.com

📞 (83) 98228-8607

📷 @josephdonald

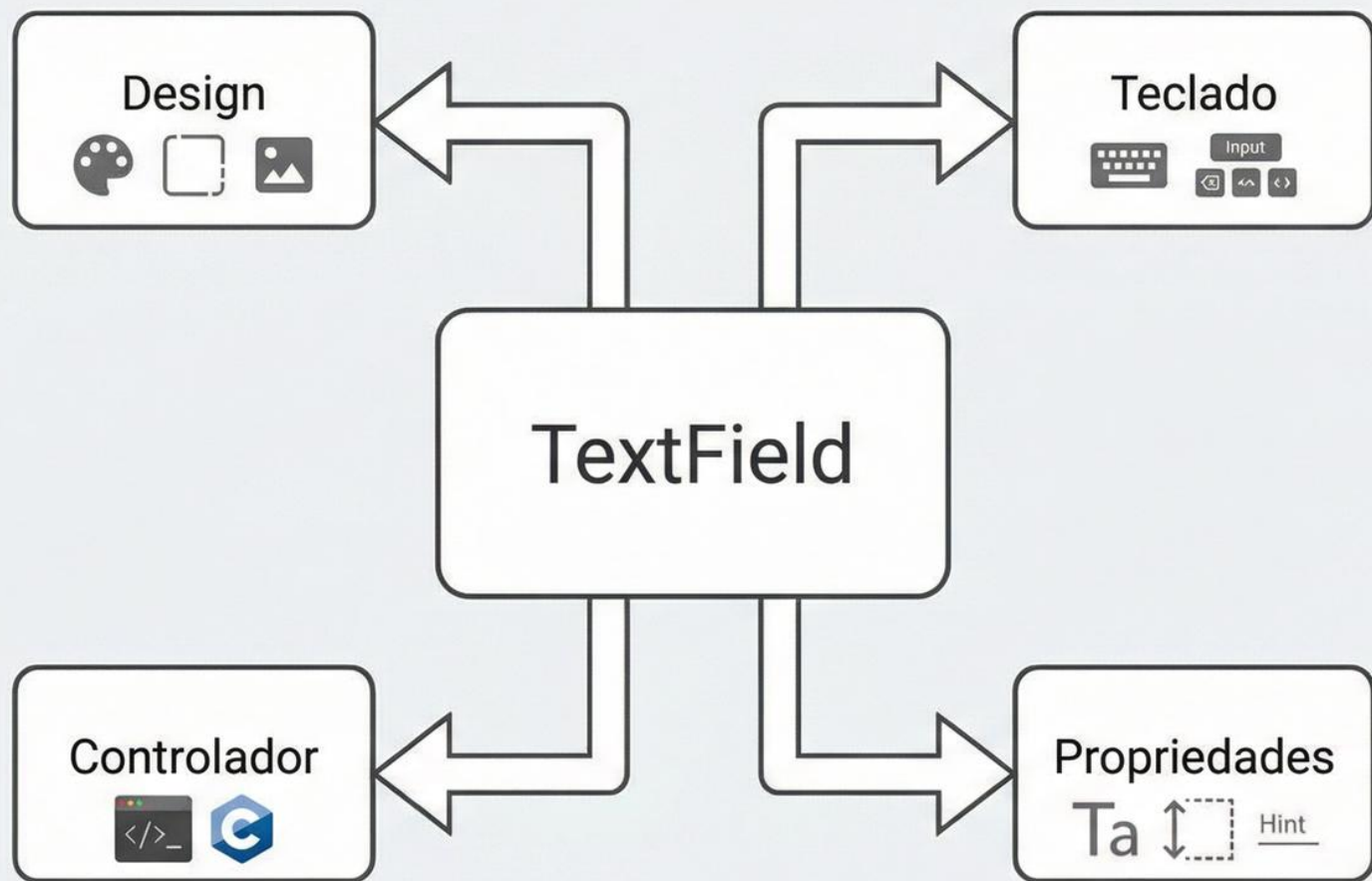
✉ 030106382@uninassau.edu.br

Introdução aos Widgets de Entrada no Flutter

- Os Widgets de entrada compõem os componentes fundamentais para a construção de interfaces interativas e dinâmicas em aplicativos móveis.
- Eles são os principais canais de comunicação entre o usuário e o sistema, permitindo a inserção de textos, números e senhas.
- No ecossistema do Flutter, esses componentes são altamente personalizáveis, adaptando-se perfeitamente aos guias de design do Material Design (Android) e Cupertino (iOS).
- A manipulação correta desses componentes é essencial para garantir uma boa experiência de usuário (UX) e uma coleta de dados eficiente e livre de erros.



1. O que é o TextField?



- O **TextField** é o widget de entrada de texto mais básico e flexível fornecido pelo Flutter.

- Ele permite a formatação rápida e direta do campo de texto, oferecendo propriedades para alterar cores, bordas, ícones e muito mais.

- É amplamente utilizado para interações isoladas e simples, onde o usuário precisa inserir uma informação pontual e sem a necessidade de um formulário complexo associado.

- Por ser um componente independente, a gestão de seu estado e o acesso aos dados digitados exigem a implementação de controladores específicos.

Exemplo de Aplicação do TextField

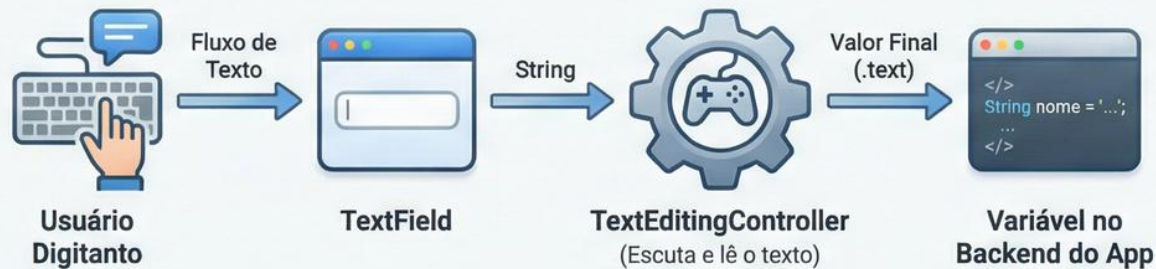
```
1  TextField(  
2    decoration: InputDecoration(  
3      border: OutlineInputBorder(),  
4      labelText: "Nome do usuário",  
5      hintText: 'Digite seu nome aqui',  
6    ), // InputDecoration  
7  ), // TextField
```

- A aplicação prática do TextField foca na customização visual para melhor aderência ao design do app.
- O uso da propriedade InputDecoration em conjunto com o OutlineInputBorder() cria uma borda retangular clássica, envolvendo todo o campo de digitação.
- Este estilo visual destaca o campo na tela, deixando claro para o usuário onde a interação deve ocorrer.
- Além da borda, é possível adicionar 'labels' (rótulos flutuantes) e 'hints' (dicas de preenchimento) para orientar o usuário sobre o dado esperado.

Nome de Usuário

Digite seu nome aqui

Capturando Texto com TextEditingController



- No Flutter, os campos de texto não mantêm o próprio estado de forma facilmente acessível sem o uso de um controlador.
- O **TextEditingController** é a classe responsável por escutar e ler o texto inserido no widget.
- Para obter ou exibir o valor final, acessa-se a propriedade `'.text'` do objeto de controle (ex: `print(nomeController.text)`).
- Também é possível usar o controlador para definir valores iniciais e limpar o campo programaticamente.
- Importante: sempre libere o controlador usando o método `dispose()` para evitar vazamentos de memória.

Principais Propriedades do TextField (Parte 1)

O TextField possui diversas propriedades para controlar a forma como o usuário insere os dados, otimizando o teclado nativo do dispositivo.

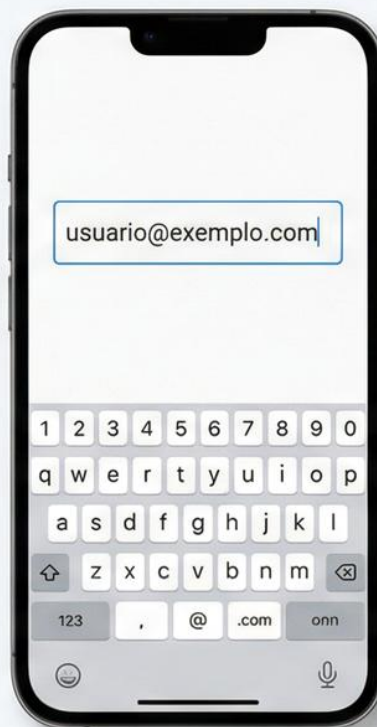
keyboardType: TextInputType.number

Força a exibição apenas do teclado numérico.



keyboardType: TextInputType.emailAddress

Adiciona o caractere '@' no teclado principal.



obscureText: true

Fundamental para campos de senha, transformando os caracteres em pontos flutuantes.



Outros tipos úteis incluem TextInputType.phone. A propriedade 'obscureText' recebe um valor booleano (true/false) e é essencial para segurança.

Principais Propriedades do TextField (Parte 2)



- Além do controle do teclado, o TextField oferece callbacks (funções de retorno) para monitorar ações em tempo real.



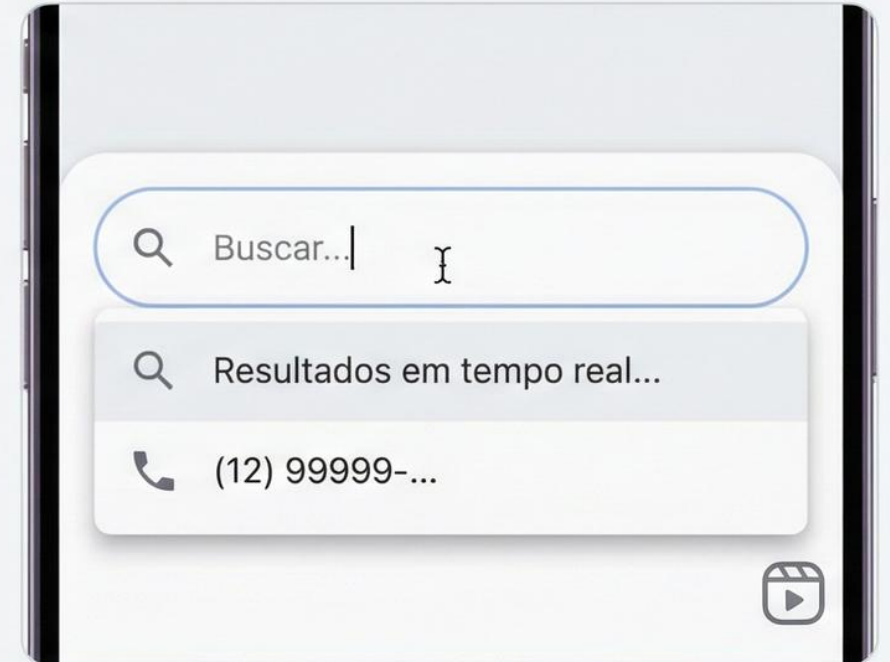
- As propriedades 'onChanged' e 'onSubmitted' permitem criar interações dinâmicas, como formatação enquanto o usuário digita ou atualização de pesquisas em tempo real.



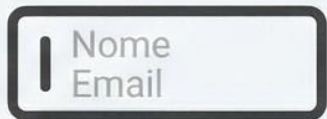
- O 'onChanged' é disparado a cada novo caractere inserido ou deletado, ideal para aplicar máscaras (ex: CPF, telefone) ou criar filtros de busca instantâneos.



- A propriedade 'maxLength' limita a quantidade máxima de caracteres que o campo pode receber, gerando também um contador visual automático no canto inferior direito.



3. O que é o TextFormField?

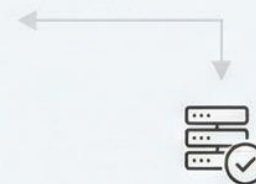
A diagram of a simple text input field. It is a rounded rectangle with a black border. Inside, there is a vertical line on the left side, and the text "Nome" is on the top line and "Email" is on the bottom line.

TextField (Simples)

- O **TextField** é um widget que envolve um **TextFormField** padrão integrando-o nativamente à classe **FormField**.



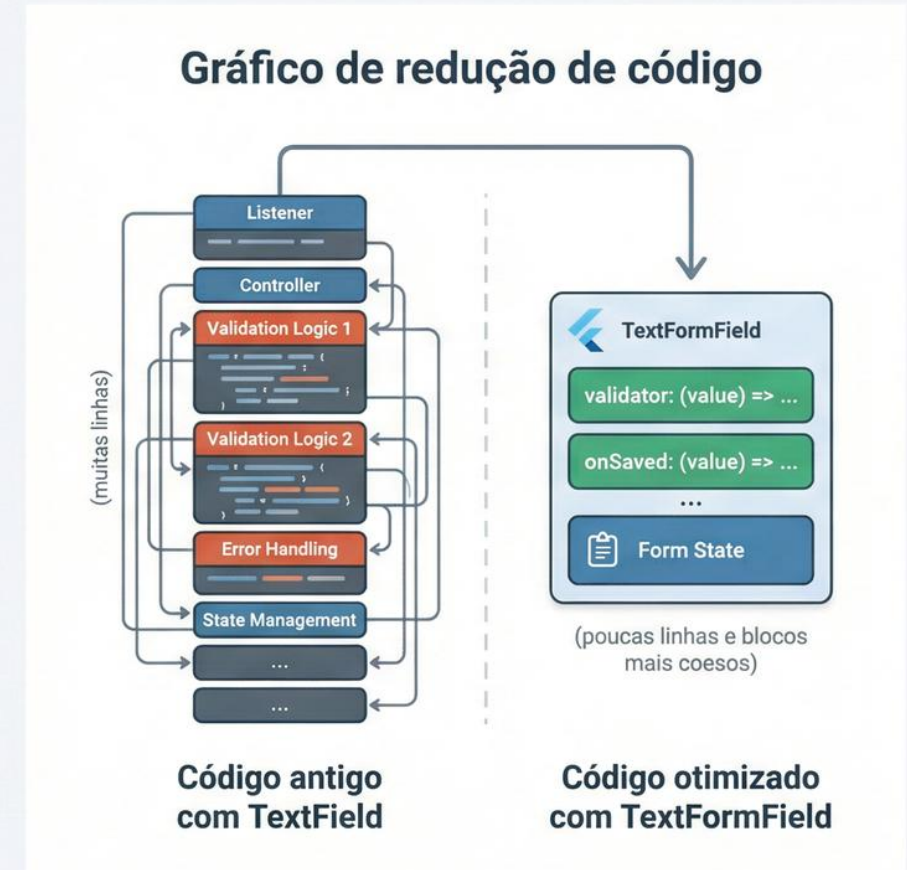
TextFormField (Robusto, com escudo de validação integrado)



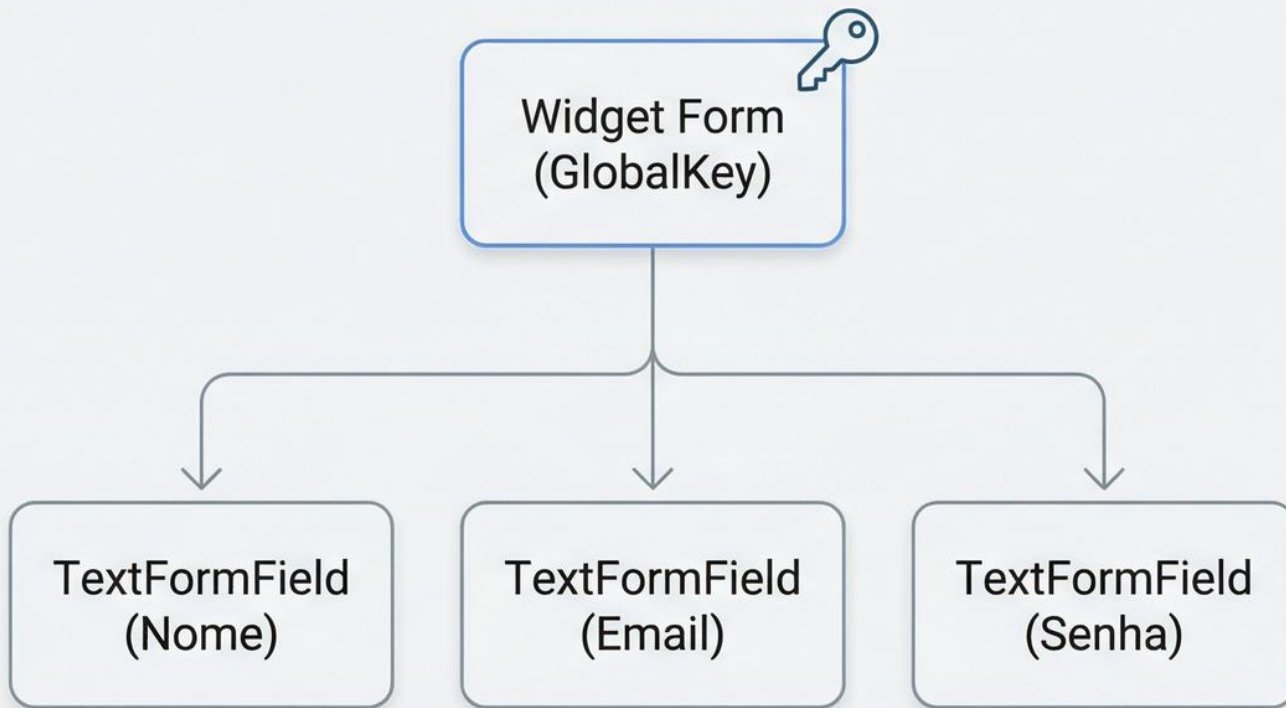
- Ele é o padrão recomendado e amplamente adotado para coleta de dados estruturados em aplicativos, como telas de cadastro e login.
- **Sua principal vantagem** é conectar a entrada de dados diretamente com o **gerenciamento de estado e validação de um formulário global**.
- Visualmente, ele é idêntico ao **TextField**, mas sob o capô, oferece um arsenal de ferramentas projetadas para validação em massa.

Funcionalidades Exclusivas do TextFormField

- O grande diferencial do **TextFormField** está na sua **capacidade de lidar com a validação de forma nativa e eficiente**.
- Ele reduz drasticamente a quantidade de código necessário para verificar erros de preenchimento, dispensando múltiplos listeners manuais.
- A propriedade "validator" aceita uma função que recebe o texto atual e retorna uma mensagem de erro caso a regra de negócio seja violada.
- A propriedade "onSaved" facilita a atribuição de todos os campos validados aos modelos de dados da aplicação em um único comando.



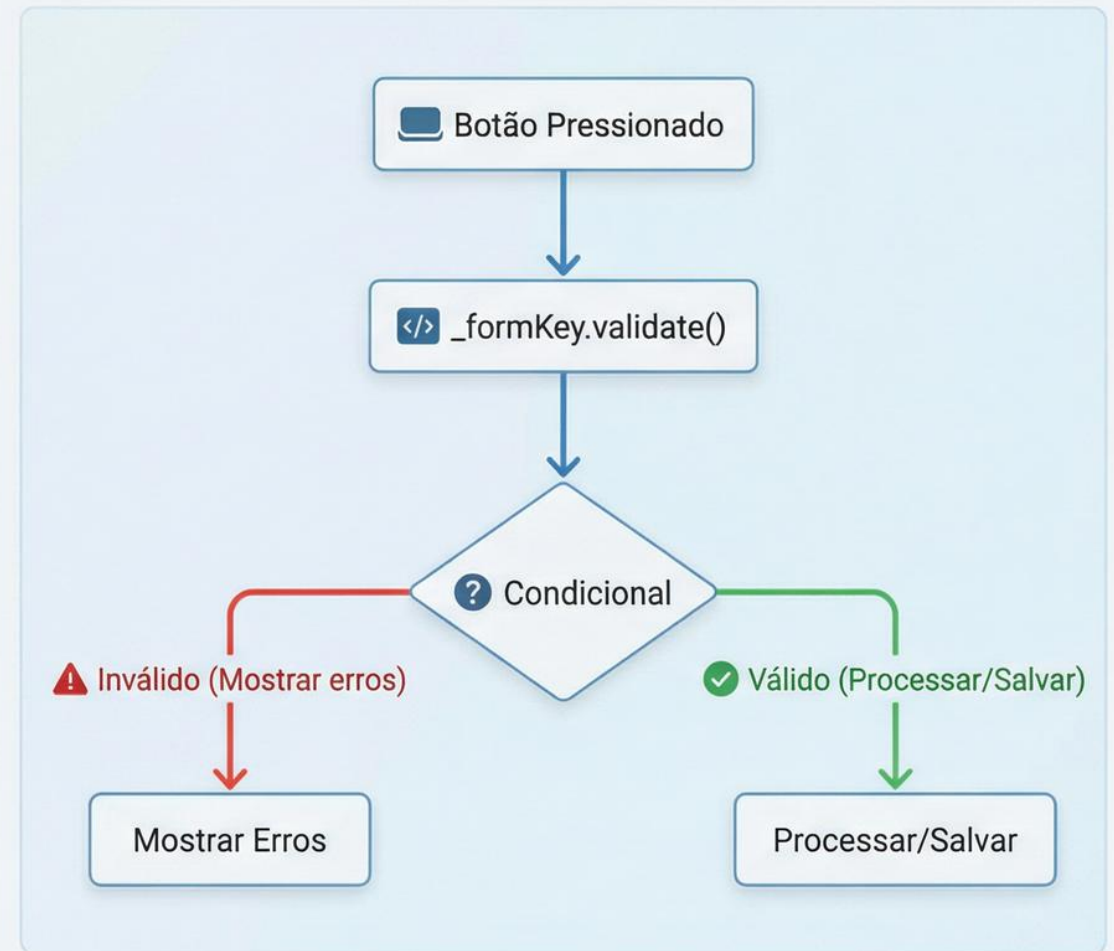
4. Estruturando Form + TextFormField



- A verdadeira potência do TextFormField é revelada quando ele é encapsulado por um widget parente chamado 'Form'.
- Os widgets TextFormField são organizados hierarquicamente como filhos deste Form, criando uma árvore de dados interconectada.
- Para controlar essa
- Para controlar essa estrutura, utilizamos uma GlobalKey<FormState>. Esta chave atua como um 'identificador único' e um 'controle remoto' do formulário.
- Ao usar a chave global, podemos validar, salvar e resetar todos os TextFormFields filhos de uma só vez, centralizando o controle do estado.

Implementando a Interface de um Formulário

- Na prática, a implementação envolve a construção da UI associando a chave ao formulário e configurando um botão de submissão.
- Ao pressionar o botão de envio (como um `ElevetaButton`), a aplicação não deve processar os dados imediatamente.
- Primeiro, chama-se o método `_formKey.currentState!.validate()` para checar os dados inseridos em todos os campos vinculados.
- Esse método dispara as funções 'validator' individuais de cada `TextFormField`. Apenas se todas retornarem sucesso (`true`), a lógica de envio prossegue.



5. Aplicando Validação de Dados

- A lógica de validação é escrita dentro da propriedade 'validator' do TextFormField.
- Esta função recebe o valor (String?) do campo no momento da validação. É ali que testamos o tamanho da string, formato de e-mail, ou se está vazio.
- Se houver algum problema (ex: campo em branco), a função deve retornar uma String contendo a mensagem de erro (ex: 'Campo obrigatório').
- Se os dados estiverem totalmente corretos e passarem nos testes, a função deve obrigatoriamente retornar 'null', indicando que não há erros associados a este campo.

```
validator: (value) {  
  if (value == null || value.isEmpty) {  
    return 'Erro';  
  }  
  return null;  
}
```


Comportamento Visual da Validação

- Quando um `TextFormField` falha na validação, o Flutter automaticamente encarrega-se de atualizar a interface.
- A borda do campo e o texto de dica ganham a cor de erro padrão (geralmente vermelho no Material Design).
- A string retornada pelo validador aparece de forma animada e elegante logo abaixo da linha do campo.
- Isso cria uma experiência incrivelmente intuitiva, orientando clara e diretamente o usuário sobre o que precisa ser corrigido sem pop-ups intrusivos.



6. Diferença: TextField vs TextFormField

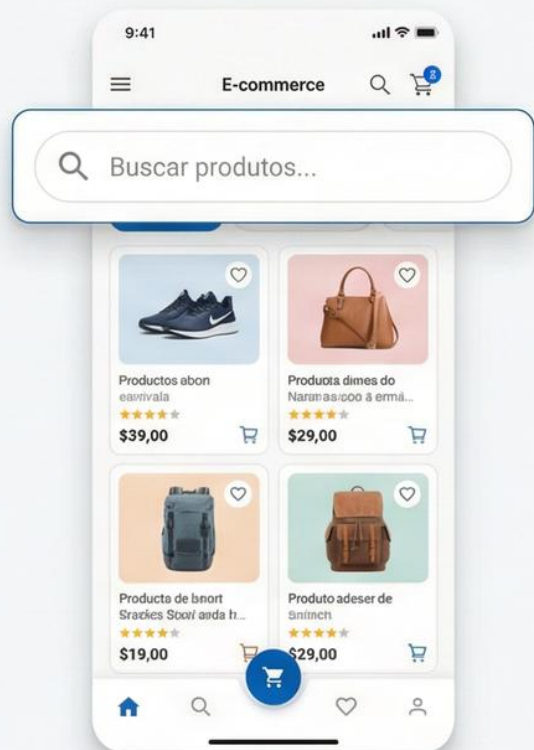
TextField	TextFormField
 Busca isolada	 Formulários
 Menor código	 Validação em massa
 Sem validador nativo	 Requer chave global

A escolha entre TextField e TextFormField depende puramente do contexto de uso no seu aplicativo. TextField é mais leve e requer menos configuração estrutural. É o componente ideal para buscar dados e capturar entradas individuais e rápidas. TextFormField é mais robusto e depende do componente Form. Ele brilha quando existem múltiplos campos e necessidade rigorosa de verificação de integridade.

Resumo: TextField para buscas ou interações isoladas; TextFormField para coletas completas de dados e validações em cadeia.

8. Quando usar o TextField?

Para consolidar o conhecimento, vamos focar nos cenários práticos e reais onde o TextField é a escolha ideal para o desenvolvedor.



Campos de busca rápida e barras de pesquisa

- Campos de busca rápida e barras de pesquisa no topo de listagens, onde a digitação dispara consultas diretas à API ou ao banco de dados local.

Interfaces de envio em chats e mensagens

- Interfaces de envio em chats e aplicativos de mensagens, onde a validação estruturada em grupo não faz sentido.
- Nesses casos, a performance e a simplicidade de instanciar apenas o widget e o TextEditingController falam mais alto.



9. Quando usar o TextFormField?

The image displays two overlapping mobile application screens. The background screen is titled 'Login Seguro' and features two TextFormField inputs: 'E-mail' with the value 'usuario@email.com' and a green checkmark, and 'Senha' with masked characters and a toggle icon. A blue 'Entrar' button is at the bottom. The foreground screen is titled 'Cadastro de Novo Usuário' and contains three TextFormField inputs: 'Nome Completo' with 'Nome Sobrenome', 'Data de Nascimento' with a date picker set to '15/08/1990', and 'CPF' with the value '000.000.000-00' and a green checkmark. A grey 'Finalizar Cadastro' button is at the bottom.

- Por outro lado, TextFormField deve ser escolhido sempre que a integridade dos dados inseridos for crucial para a regra de negócio e submissão.
- Exemplos práticos de uso incluem:
 - Telas de login seguro, exigindo que e-mail e senha obedecem a formatos estritos antes de acionar a autenticação.
 - Cadastros de novos usuários, formulários de endereços (com validação de CEP e estado), e formulários de pesquisa complexos de mercado.
 - Nestas situações, impedir a inserção de dados equivocados na fonte (o dispositivo do usuário) poupa banda, processamento do servidor e melhora a experiência.

Desafio Prático 1 – Calculadora de IMC



$$\text{IMC} = \frac{\text{Peso } \text{kg}}{\text{Altura} * \text{Altura } \text{m}}$$

Classificação	Faixa de IMC
Abaixo do peso	< 18.5
Normal	18.5–24.9
Sobrepeso	25–29.9
Obesidade	≥ 30

- 📁 Vamos colocar os conhecimentos em prática!
O primeiro desafio consiste em criar uma Calculadora de Índice de Massa Corporal (IMC).
- 📝 Você precisará de TextFields ou TextFormFields para capturar o Peso (em kg) e a Altura (em metros), garantindo o uso de teclado numérico (TextInputType.number).
- ☑️ Crie um botão que realize o cálculo matemático: $\text{IMC} = \text{Peso} / (\text{Altura} * \text{Altura})$.
- ① Mostre o resultado e a respectiva classificação: Abaixo do peso (< 18.5), Normal (18.5–24.9), Sobrepeso (25–29.9) ou Obesidade (≥ 30).

Desafio Prático 2 – Conversor de Temperatura

O segundo desafio exigirá atenção à gestão de estado. Crie um aplicativo para conversão de temperaturas.



- Forneça um único campo para o usuário inserir uma temperatura em graus Celsius (°C). Aplique a lógica:
- **Fahrenheit** = $(\text{Celsius} * 9/5) + 32$
- **Kelvin** = $\text{Celsius} + 273.15$
- Saída Esperada: O aplicativo deve exibir os dois resultados (Fahrenheit e Kelvin) simultaneamente na tela após o usuário pressionar o botão 'Converter'.